R.B GOTHI JAIN COLLEGE FOR WOMEN

TEAM MEMBERS :
1.Pavithra G (Team Leader)
2.Dilli Rani B
3.Hemanuja K
4.Shalini E
5.Nasreen Shifana A

AI WeatherWise - Deliver intelligent weather forecasts and insights.

Description

The AI WeatherWise is a robust RESTful backend platform built with **Node.js** and **Express.js**, utilizing **MongoDB** and **Mongoose ODM** for data persistence. It delivers real-time weather metrics integrated with **Google Gemini AI** to provide intelligent summaries and personalized recommendations based on live environmental conditions.

The platform features secure JWT authentication, bcrypt password hashing, and a comprehensive suite of endpoints for managing favorite locations and fetching weather insights. It includes a resilient fallback mode ensuring API functionality even without external API keys, alongside centralized error handling and request sanitization.

Scenario-Based Case Study

Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather conditions and receive AI-driven activity suggestions based on current forecasts.

Problem

- Difficulty tracking and managing multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural language weather summaries rather than just raw data metrics.
- Absence of a secure system to store and sync preferences across devices.

Solution

The AI WeatherWise provides a centralized service that handles weather data retrieval and AI orchestrations. It manages a secure User and Location database, allowing for customized insights and automated recommendation workflows through Google Gemini.

Usage

- **User Usage:** Manages favorite cities (CRUD), fetches real-time weather, and requests AI insights for recommendations and summaries.
- **Developer/Admin Usage:** Monitors API health, manages global user registrations, and verifies weather/AI service integrations.
- **Weather Application User:** Interfaces with the public weather endpoints to consume current metrics without requiring an account.

Outcome

By leveraging AI and centralized data management, users spend less time deciphering raw metrics. The system ensures data integrity via Mongoose schemas and provides high-value, context-aware weather advice through Gemini integrations.

1. Software Requirements

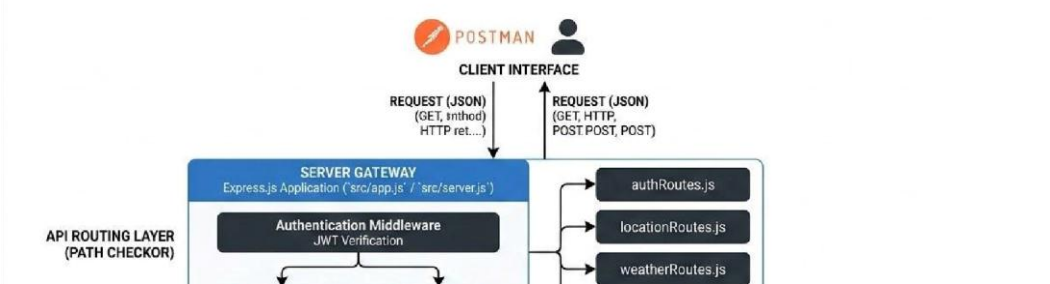
- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Gemini AI SDK:** Integration using @google/generative-ai for natural language insights.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Epic 1: PROJECT ARCHITECTURE

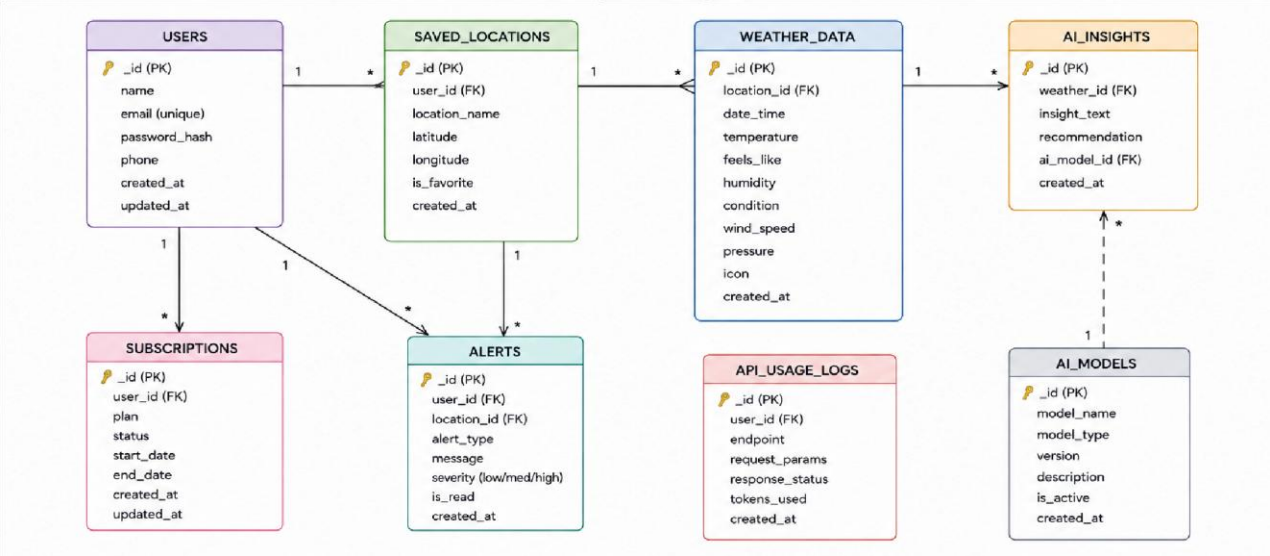
The AI WeatherWise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages User and Location entities.



Components Definition

- **Interface:** API tools like Postman query distinct route paths to extract JSON entities.
- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.
- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.
- **AI Service Integration:** Leverages @google/generative-ai to generate weather summaries and personalized recommendations based on live temperature and conditions.

Entity-Relationship (ER) Diagram Description



The data architecture manages User profiles and their associated favorite Locations through logical references:

1. User Entity

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Location Entity

- _id (Primary Key - ObjectId)
- user (Foreign Key referencing User Collection)
- city (String, Required)
- country (String, Required)
- createdAt (Date, Default: Now) **Key Structural Mappings**
- **User to Location (One-to-Many):** A registered user can save multiple favorite locations.

Key Features

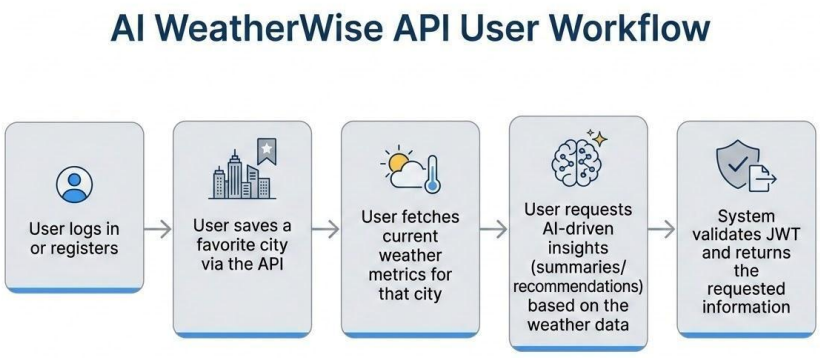
- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths.
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability.
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.

Roles and Responsibilities

- **Admin:**
  - Responsible for user account management, including creating, deleting, or suspending accounts.
  - ○ Oversight of API health and system logs to monitor performance. ○ Monitoring and managing database integrity.
- **Registered User:**
  - Responsible for creating and maintaining a list of favorite locations.
  - ○ Initiating requests for real-time weather data.
  - ○ Interacting with the Gemini AI service for personalized weather summaries and activity recommendations.
  - ○ Managing their own authentication credentials, such as updating profile details.

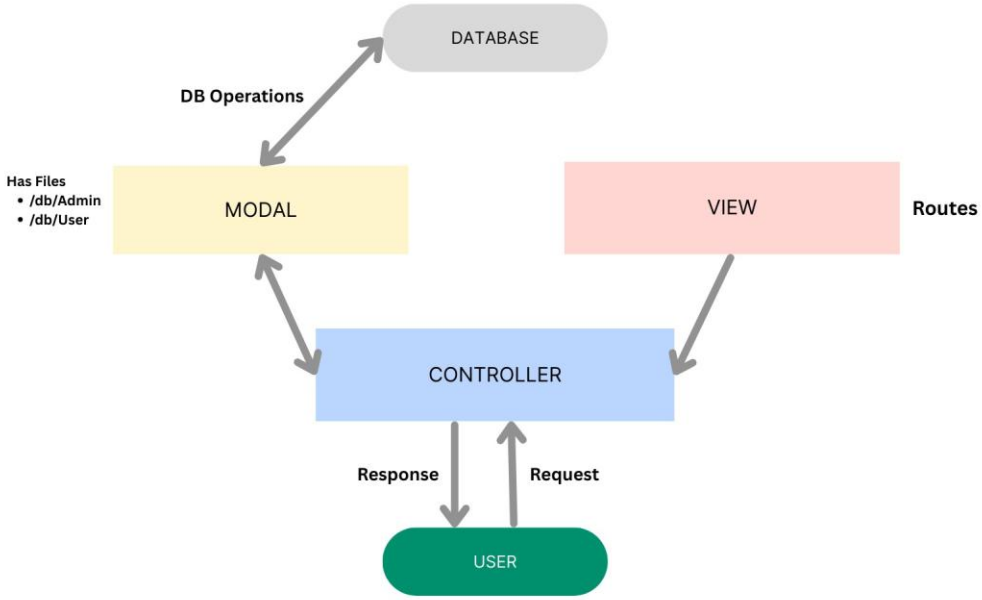
User Flow:

1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.
5. System validates JWT and returns the requested information.



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer**. It links network requests on specified paths directly to their associated controller handler logic maps.

Epic 2: Project Setup and Configuration

Creating the Project Folder

- Create a new folder named **AI Weatherwise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

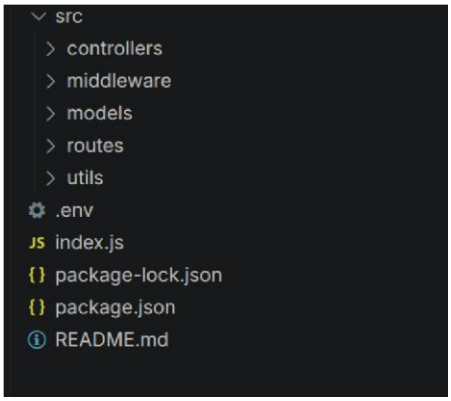
Install Nodemon for development:

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

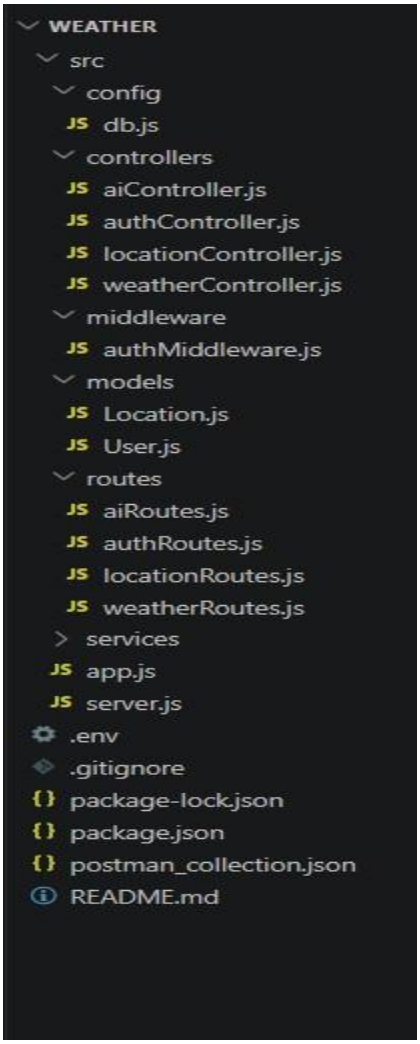
Create a **.env** file in the project root and configure the required environment variables.

PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret GEMINI_API_KEY=your_gemini_api_key

Epic 3: Backend Development

The AI weatherwise backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.

Backend Structure:



Core System Script Implementations

1. Application Entry Point (index.js)

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
    });
  })
  .catch((error) => {
    console.error('Database connection failed. Express server aborted:', error.message);
    process.exit(1);
  });
```

Epic4: Database Configuration

API Testing Demo :

1. Configure MongoDB:

- Install Mongoose.
  In your Node.js project directory, run:
  npm install mongoose ●

Create database connection.

- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

Location Schema:

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

User schema:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Epic 5: Project Executions:

The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client. **Running the Backend Server**

Install all required dependencies:

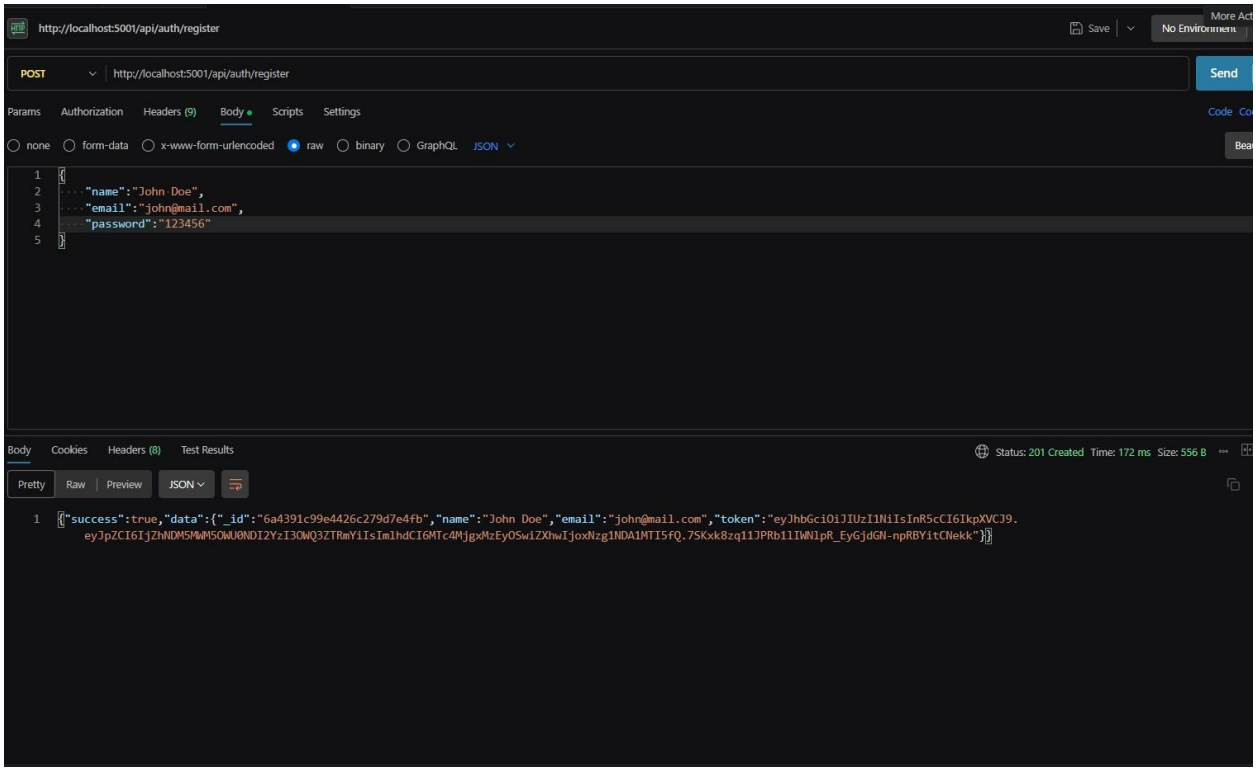
```
npm install
```

Start the development server:

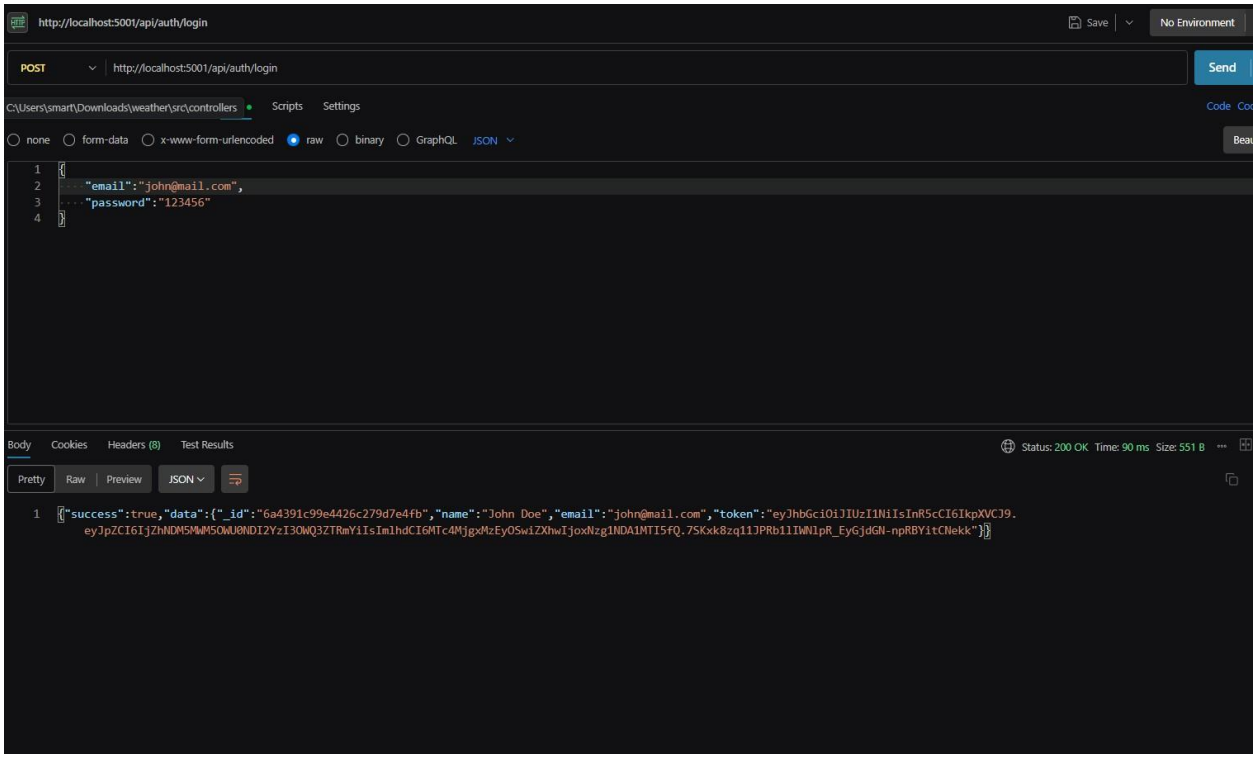
```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

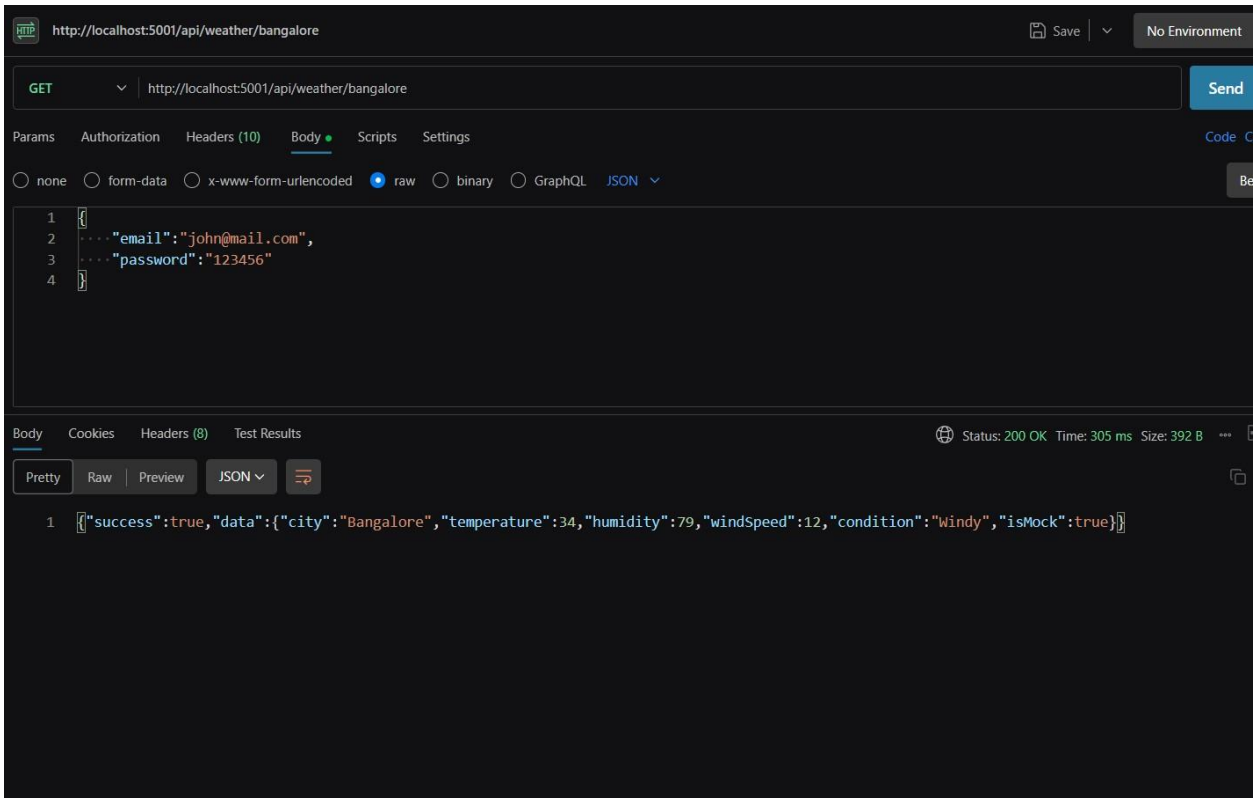
API testing: Register API:



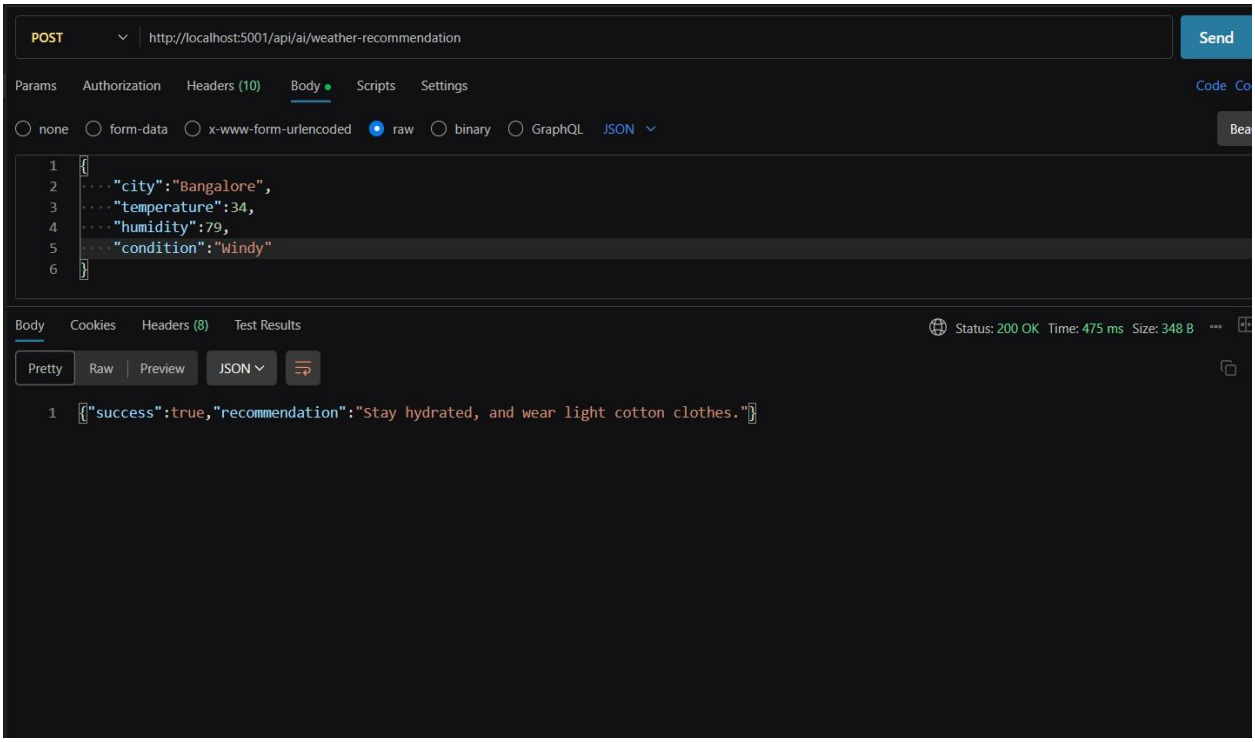
Login API:



Weather API:



Recommendation API:


POST
http://localhost:5001/api/ai/weather-recommendation
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"city":"Bangalore",
"temperature":34,
"humidity":79,
"condition":"Windy"
}
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 475 ms
Size: 348 B
Pretty
Raw
Preview
JSON
{"success":true,"recommendation":"Stay hydrated, and wear light cotton clothes."}